



Name : Postiwala Vatsal

PRN No: 26070122509

Batch : B3

ASSIGNMENT No: 6

TITLE: Write a Java program to demonstrate the use of an inner class, an anonymous class.

Problem Statement

Write a Java program to demonstrate the use of an inner class, an anonymous class.

Learning Objectives

To implement inner classes and anonymous classes for improving encapsulation and event handling in Java.

Theory

An inner class is a class defined inside another class and has access to all members of the enclosing class, including private members. Inner classes help improve code organization and logical grouping of related classes. Anonymous classes are unnamed inner classes that are instantiated only once and are mainly used for implementing interfaces or abstract classes. They simplify code by eliminating the need for separate class definitions. These concepts are widely used in Java GUI programming and event handling.

Code

```
class College{

    String CollegeName = "SIT Pune";

    class Student{

        void display(){
            System.out.println("Student belongs to "+CollegeName);
        }
    }
}

interface Greeting{
    void Sayhello();
}

public class InnerAnonymousDemo{
    public static void main(String[] args) {

        College c = new College();
        College.Student s = c.new Student();
    }
}
```

```

s.display();

Greeting g = new Greeting(){

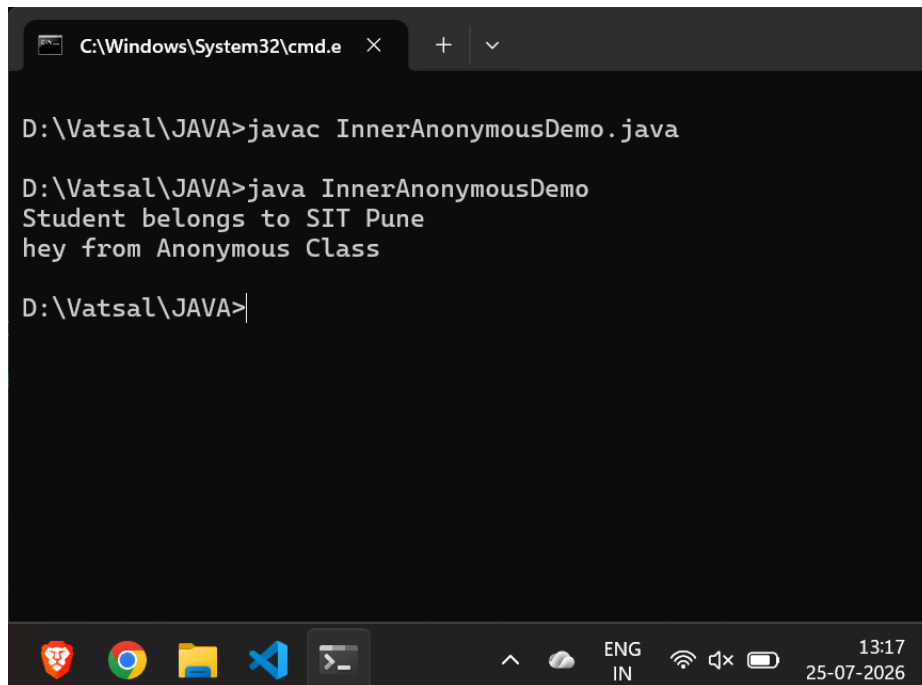
    public void Sayhello(){
        System.out.println("hey from Anonymous Class");
    }

};

g.Sayhello();
}
}

```

Output



```

C:\Windows\System32\cmd.e
D:\Vatsal\JAVA>javac InnerAnonymousDemo.java

D:\Vatsal\JAVA>java InnerAnonymousDemo
Student belongs to SIT Pune
hey from Anonymous Class

D:\Vatsal\JAVA>

```

Exercise

1. Create a **Vehicle program** where an inner class displays vehicle details and anonymous class performs an action.

Code

```
interface VehicleAction {
    void performAction();
}

class Vehicle {
    String name = "Honda";
    String model = "Civic";

    class VehicleDetails {
        void display() {
            System.out.println("--- Vehicle Details ---");
            System.out.println("Name: " + name);
            System.out.println("Model: " + model);
        }
    }
}

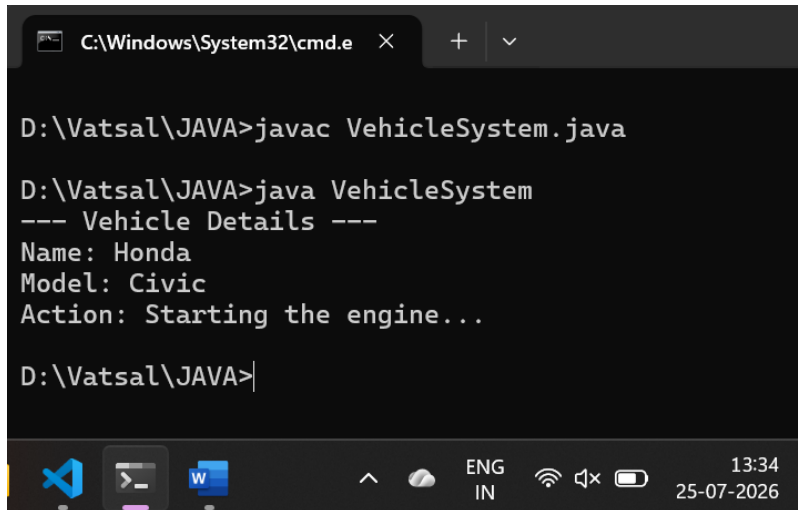
public class VehicleSystem {
    public static void main(String[] args) {

        Vehicle v = new Vehicle();
        Vehicle.VehicleDetails details = v.new VehicleDetails();
        details.display();

        VehicleAction action = new VehicleAction() {
            public void performAction() {
                System.out.println("Action: Starting the engine...");
            }
        };

        action.performAction();
    }
}
```

Output

A screenshot of a Windows command prompt window. The title bar shows 'C:\Windows\System32\cmd.e' with a close button and window controls. The command prompt shows the following text:

```
D:\Vatsal\JAVA>javac VehicleSystem.java

D:\Vatsal\JAVA>java VehicleSystem
--- Vehicle Details ---
Name: Honda
Model: Civic
Action: Starting the engine...

D:\Vatsal\JAVA>|
```

The taskbar at the bottom shows icons for Visual Studio Code, the command prompt, and Word. System tray icons include network, volume, and battery. The time is 13:34 and the date is 25-07-2026.

2. Develop a **Food Delivery Application** where an inner class handles order details and anonymous classes handle delivery status updates.

Code

```
interface DeliveryStatus {
    void updateStatus();
}

class FoodDeliveryApp {
    String restaurantName = "Pizza Planet";

    class OrderDetails {
        int orderId;
        String item;

        OrderDetails(int orderId, String item) {
            this.orderId = orderId;
            this.item = item;
        }

        void displayOrder() {
            System.out.println("--- Order Details ---");
            System.out.println("Order ID: " + orderId);
            System.out.println("Restaurant: " + restaurantName);
            System.out.println("Item Ordered: " + item);
        }
    }
}
```

```

public class FoodDelivery {
    public static void main(String[] args) {

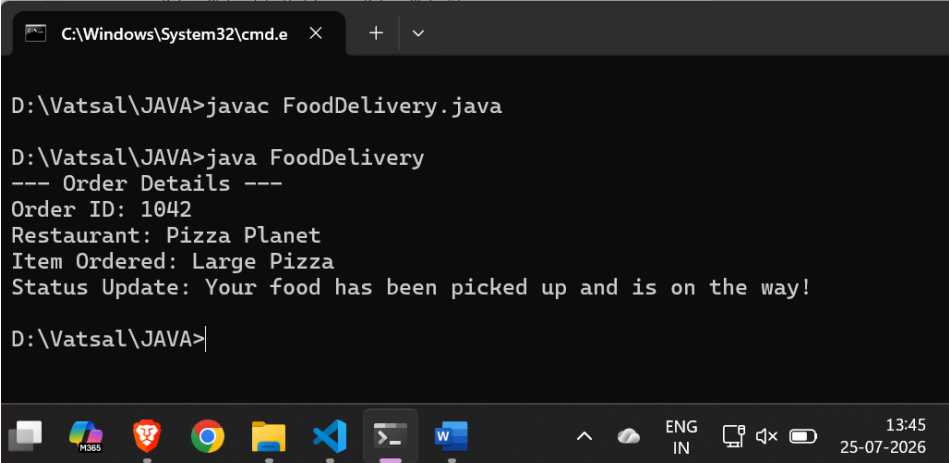
        FoodDeliveryApp app = new FoodDeliveryApp();
        FoodDeliveryApp.OrderDetails order = app.new OrderDetails(1042, "Large Pizza");
        order.displayOrder();

        DeliveryStatus statusUpdate = new DeliveryStatus() {
            public void updateStatus() {
                System.out.println("Status Update: Your food has been picked up and is on the way!");
            }
        };

        statusUpdate.updateStatus();
    }
}

```

Output



```

C:\Windows\System32\cmd.e x + v

D:\Vatsal\JAVA>javac FoodDelivery.java

D:\Vatsal\JAVA>java FoodDelivery
--- Order Details ---
Order ID: 1042
Restaurant: Pizza Planet
Item Ordered: Large Pizza
Status Update: Your food has been picked up and is on the way!

D:\Vatsal\JAVA>

```

Conclusion

- **Inner Classes** (Student, VehicleDetails, OrderDetails) are best used for logically grouping helper classes that need direct access to the outer class's fields and methods.
- **Anonymous Classes** (Greeting, VehicleAction, DeliveryStatus) are ideal for lightweight, one-off implementations of interfaces without the overhead of creating dedicated .java files.